Ex No: 14

Log Analysis for detection and response

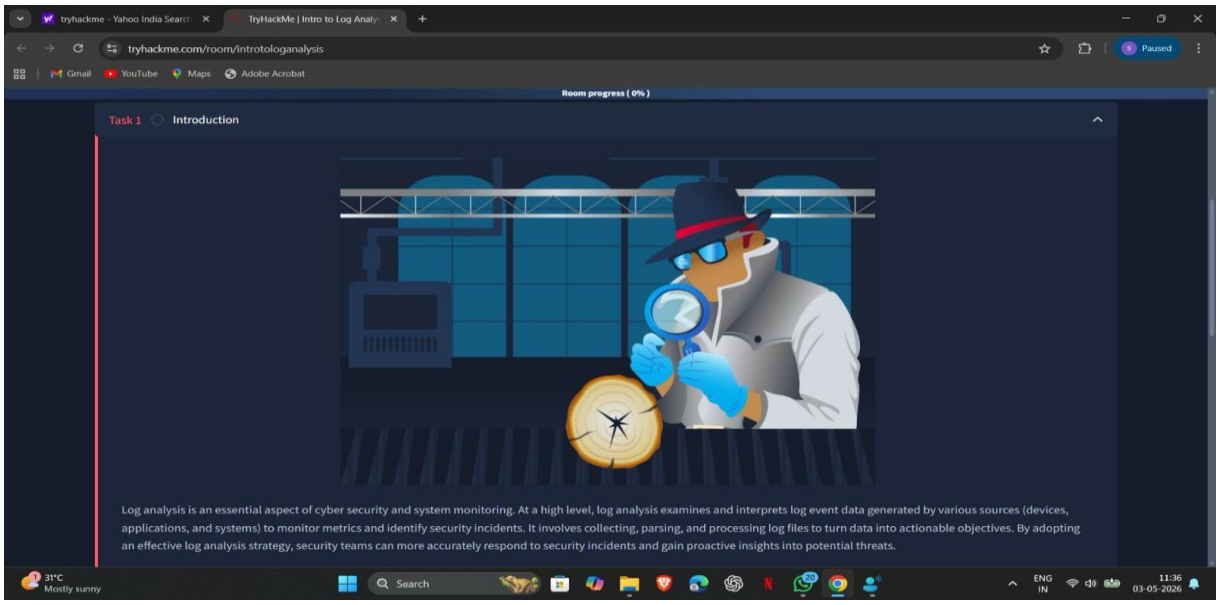
Aim:

To analyze system and network logs to detect suspicious activities and respond to security incidents effectively in a SOC environment.

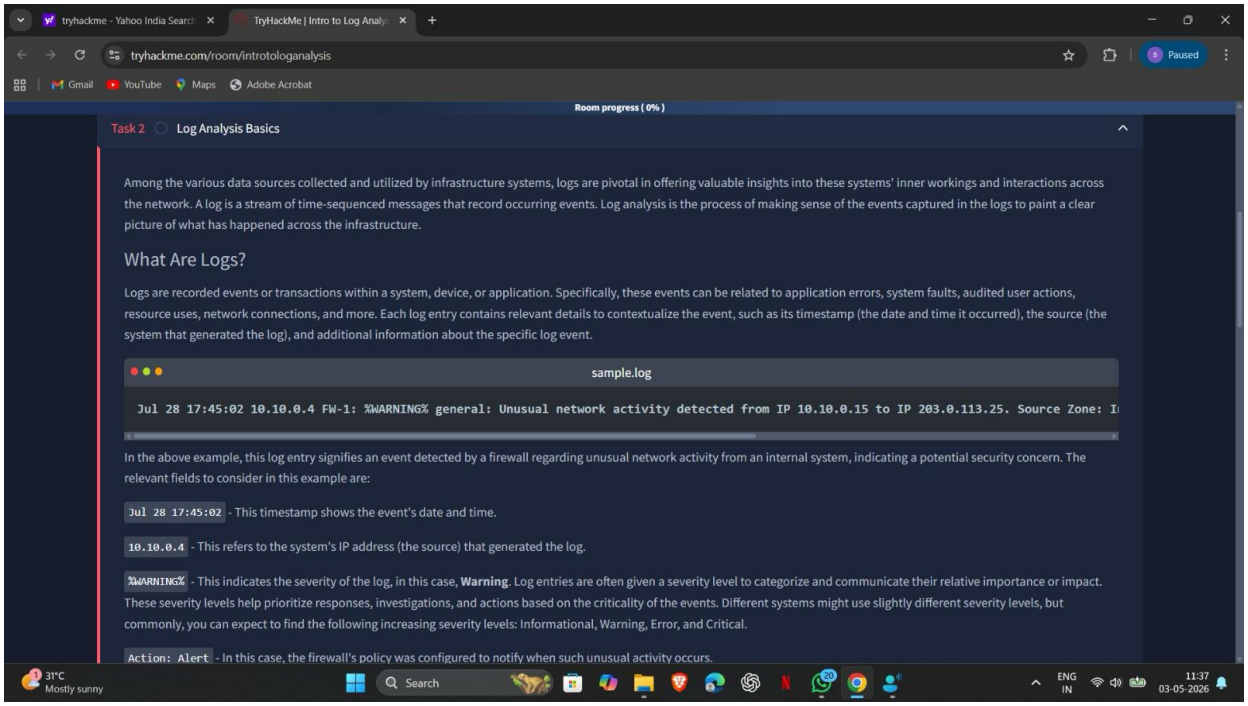
Procedure:

1. Open a web browser and go to TryHackMe, then log in with your account.
2. Search for the room related to Log Analysis for Detection and Response and open it.
3. Read the introduction to understand the importance of log analysis in cybersecurity and incident response.
4. Start the room by clicking “Start” or launch the machine if required.
5. Study different types of logs such as system logs, network logs, authentication logs, and application logs.
6. Analyze the provided log data to identify unusual or suspicious activities like failed login attempts, unauthorized access, or abnormal traffic.
7. Use filtering, searching, and correlation techniques to find Indicators of Compromise (IoCs).
8. Answer all the questions in each task based on your findings and verify the answers.
9. Document key observations, tools used, and take screenshots if required for record submission.
10. Complete all tasks until the room shows 100% completion.
11. Log out from TryHackMe after finishing the experiment.

Task 1: Introduction



Task 2: Log analysis basics



Among the various data sources collected and utilized by infrastructure systems, logs are pivotal in offering valuable insights into these systems' inner workings and interactions across the network. A log is a stream of time-sequenced messages that record occurring events. Log analysis is the process of making sense of the events captured in the logs to paint a clear picture of what has happened across the infrastructure.

What Are Logs?

Logs are recorded events or transactions within a system, device, or application. Specifically, these events can be related to application errors, system faults, audited user actions, resource uses, network connections, and more. Each log entry contains relevant details to contextualize the event, such as its timestamp (the date and time it occurred), the source (the system that generated the log), and additional information about the specific log event.

sample.log

```
Jul 28 17:45:02 10.10.0.4 FW-1: %WARNING% general: Unusual network activity detected from IP 10.10.0.15 to IP 203.0.113.25. Source Zone: I
```

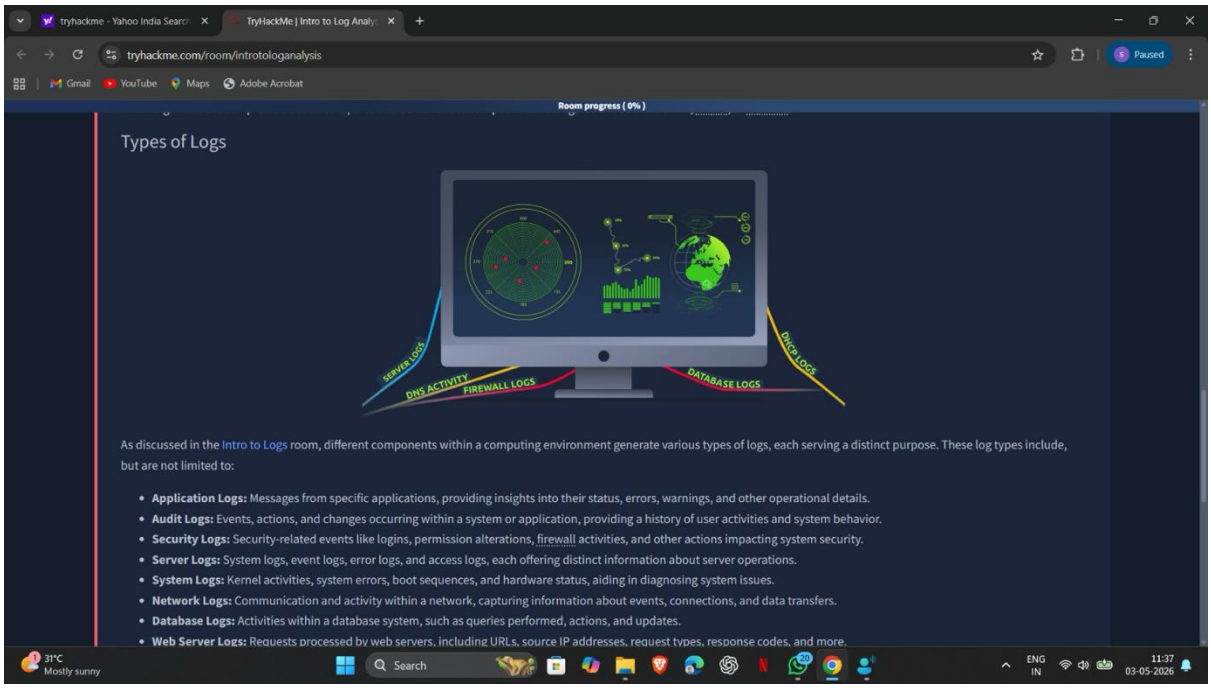
In the above example, this log entry signifies an event detected by a firewall regarding unusual network activity from an internal system, indicating a potential security concern. The relevant fields to consider in this example are:

`Jul 28 17:45:02` - This timestamp shows the event's date and time.

`10.10.0.4` - This refers to the system's IP address (the source) that generated the log.

`%WARNING%` - This indicates the severity of the log, in this case, **Warning**. Log entries are often given a severity level to categorize and communicate their relative importance or impact. These severity levels help prioritize responses, investigations, and actions based on the criticality of the events. Different systems might use slightly different severity levels, but commonly, you can expect to find the following increasing severity levels: Informational, Warning, Error, and Critical.

`Action: Alert` - In this case, the firewall's policy was configured to notify when such unusual activity occurs.

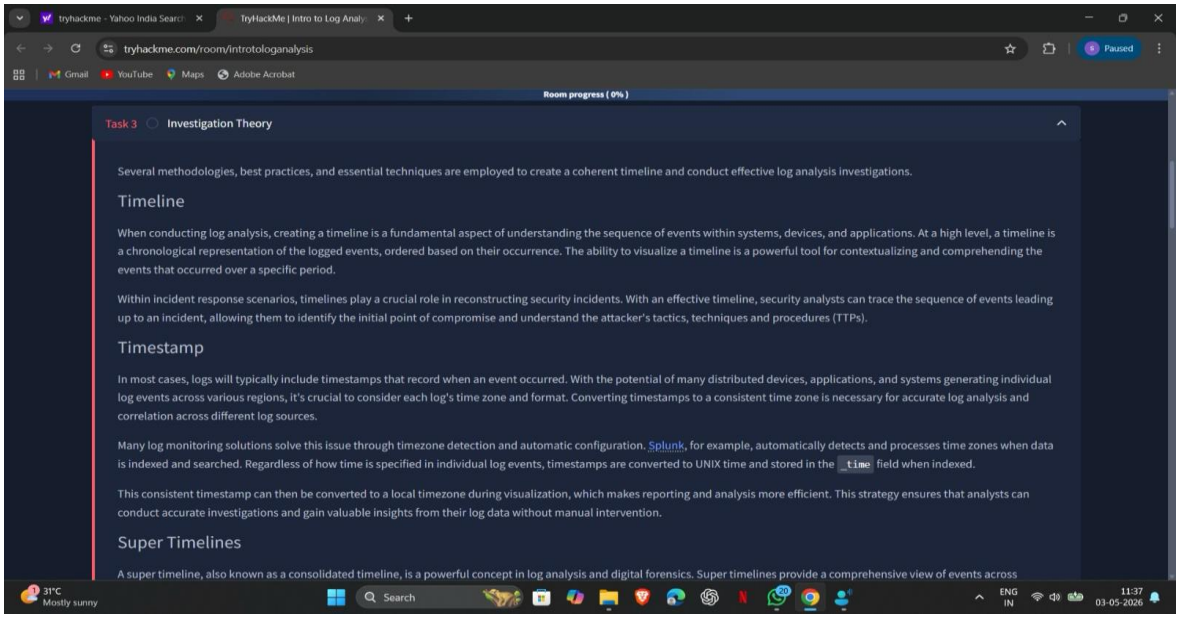


Types of Logs

As discussed in the Intro to Logs room, different components within a computing environment generate various types of logs, each serving a distinct purpose. These log types include, but are not limited to:

- **Application Logs:** Messages from specific applications, providing insights into their status, errors, warnings, and other operational details.
- **Audit Logs:** Events, actions, and changes occurring within a system or application, providing a history of user activities and system behavior.
- **Security Logs:** Security-related events like logins, permission alterations, firewall activities, and other actions impacting system security.
- **Server Logs:** System logs, event logs, error logs, and access logs, each offering distinct information about server operations.
- **System Logs:** Kernel activities, system errors, boot sequences, and hardware status, aiding in diagnosing system issues.
- **Network Logs:** Communication and activity within a network, capturing information about events, connections, and data transfers.
- **Database Logs:** Activities within a database system, such as queries performed, actions, and updates.
- **Web Server Logs:** Requests processed by web servers, including URLs, source IP addresses, request types, response codes, and more.

Task 3: Investigation theory



Room progress (0%)

Task 3 Investigation Theory

Several methodologies, best practices, and essential techniques are employed to create a coherent timeline and conduct effective log analysis investigations.

Timeline

When conducting log analysis, creating a timeline is a fundamental aspect of understanding the sequence of events within systems, devices, and applications. At a high level, a timeline is a chronological representation of the logged events, ordered based on their occurrence. The ability to visualize a timeline is a powerful tool for contextualizing and comprehending the events that occurred over a specific period.

Within incident response scenarios, timelines play a crucial role in reconstructing security incidents. With an effective timeline, security analysts can trace the sequence of events leading up to an incident, allowing them to identify the initial point of compromise and understand the attacker's tactics, techniques and procedures (TTPs).

Timestamp

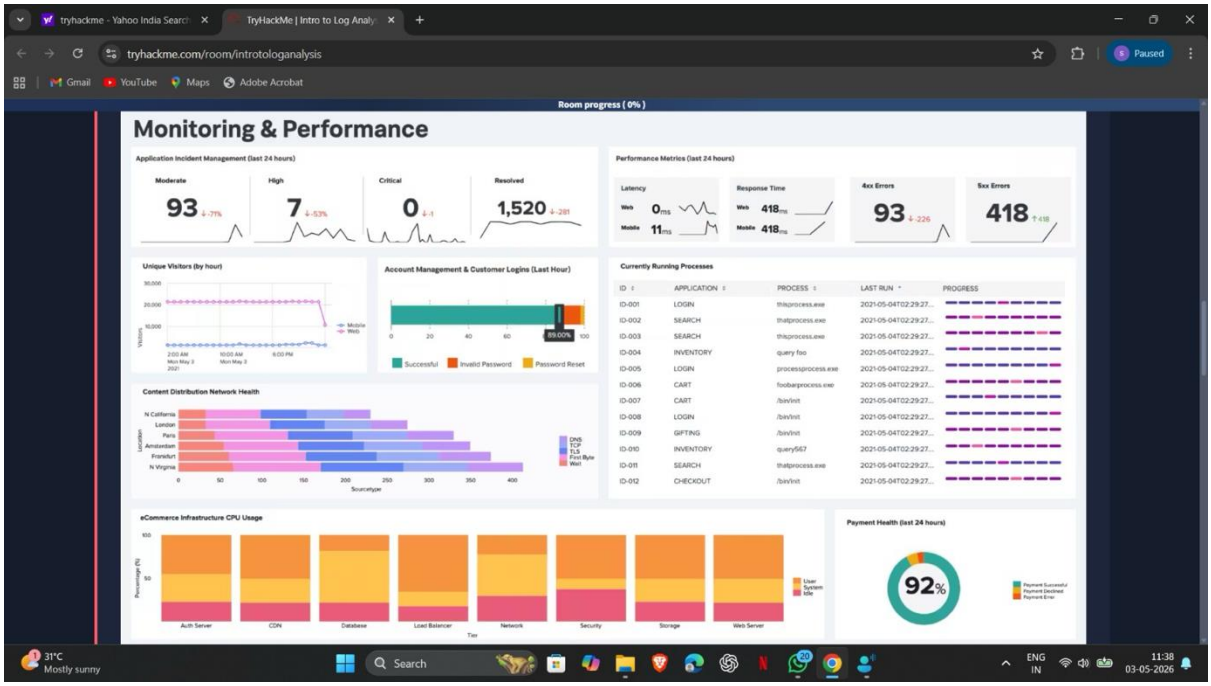
In most cases, logs will typically include timestamps that record when an event occurred. With the potential of many distributed devices, applications, and systems generating individual log events across various regions, it's crucial to consider each log's time zone and format. Converting timestamps to a consistent time zone is necessary for accurate log analysis and correlation across different log sources.

Many log monitoring solutions solve this issue through timezone detection and automatic configuration. Splunk, for example, automatically detects and processes time zones when data is indexed and searched. Regardless of how time is specified in individual log events, timestamps are converted to UNIX time and stored in the `_time` field when indexed.

This consistent timestamp can then be converted to a local timezone during visualization, which makes reporting and analysis more efficient. This strategy ensures that analysts can conduct accurate investigations and gain valuable insights from their log data without manual intervention.

Super Timelines

A super timeline, also known as a consolidated timeline, is a powerful concept in log analysis and digital forensics. Super timelines provide a comprehensive view of events across



Task 4: Detection Engineering

tryhackme - Yahoo India Search | TryHackMe | Intro to Log Analy

tryhackme.com/room/introtologanalysis

Gmail YouTube Maps Adobe Acrobat

Room progress (0%)

Task 4 Detection Engineering

Common Log File Locations

A crucial aspect of log analysis is understanding where to locate log files generated by various applications and systems. While log file paths can vary due to system configurations, software versions, and custom settings, knowing common log file locations is essential for efficient investigation and threat detection.

- **Web Servers:**
  - **Nginx:**
    - Access Logs: `/var/log/nginx/access.log`
    - Error Logs: `/var/log/nginx/error.log`
  - **Apache:**
    - Access Logs: `/var/log/apache2/access.log`
    - Error Logs: `/var/log/apache2/error.log`
- **Databases:**
  - **MySQL:**
    - Error Logs: `/var/log/mysql/error.log`
  - **PostgreSQL:**
    - Error and Activity Logs: `/var/log/postgresql/postgresql-{version}-main.log`
- **Web Applications:**
  - **PHP:**
    - Error Logs: `/var/log/php/error.log`
- **Operating Systems:**

31°C Mostly sunny Search ENG IN 11:38 03-05-2026

tryhackme - Yahoo India Search | TryHackMe | Intro to Log Analy

tryhackme.com/room/introtologanalysis

Gmail YouTube Maps Adobe Acrobat

Room progress (0%)

Exploiting cross-site scripting (XSS) vulnerabilities allow attackers to inject malicious scripts into web pages. To identify common XSS attack patterns, it is often helpful to look for log entries with unexpected or unusual input that includes script tags (`<script>`) and event handlers (`onmouseover` , `onclick` , `onerror`). A useful XSS payload list to reference can be found here.

In the example below, a cross-site scripting attempt can be identified by the `<script>alert(1);</script>` payload inserted into the `search` parameter, which is a common testing method for XSS vulnerabilities.

xss.log

```
10.10.19.31 - - [2023-08-04 16:12:11] "GET /products.php?search=<script>alert(1);</script> HTTP/1.1" 200 5153
```

Path Traversal

Exploiting path traversal vulnerabilities allows attackers to access files and directories outside a web application's intended directory structure, leading to unauthorized access to sensitive files or code. To identify common traversal attack patterns, look for traversal sequence characters (`../` and `../../`) and indications of access to sensitive files (`/etc/passwd` , `/etc/shadow`). A useful directory traversal payload list to reference can be found here.

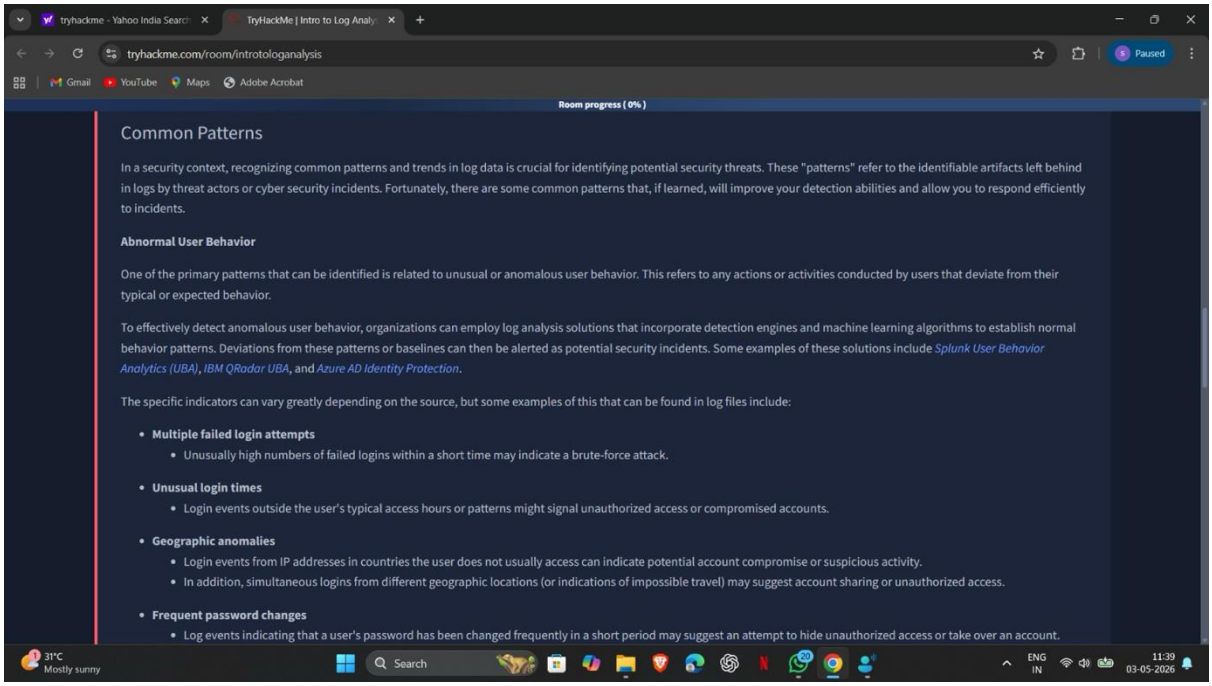
It is important to note, like with the above examples, that directory traversals are often URL encoded (or double URL encoded) to avoid detection by firewalls or monitoring tools. Because of this, `%2E` and `%2F` are useful URL-encoded characters to know as they refer to the `.` and `/` respectively.

In the below example, a directory traversal attempt can be identified by the repeated sequence of `../` characters, indicating that the attacker is attempting to "back out" of the web directory and access the sensitive `/etc/passwd` file on the server.

path-traversal.log

```
10.10.113.45 - - [2023-08-05 18:17:25] "GET /../../../../etc/passwd HTTP/1.1" 200 505
```

31°C Mostly sunny Search ENG IN 11:39 03-05-2026



Common Patterns

In a security context, recognizing common patterns and trends in log data is crucial for identifying potential security threats. These "patterns" refer to the identifiable artifacts left behind in logs by threat actors or cyber security incidents. Fortunately, there are some common patterns that, if learned, will improve your detection abilities and allow you to respond efficiently to incidents.

Abnormal User Behavior

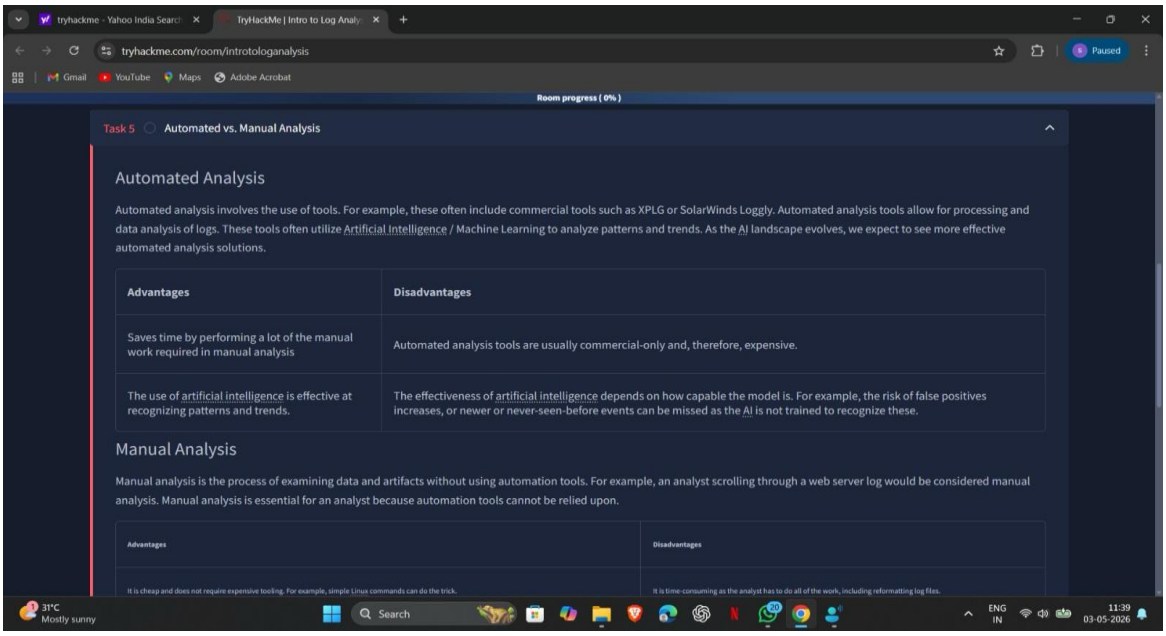
One of the primary patterns that can be identified is related to unusual or anomalous user behavior. This refers to any actions or activities conducted by users that deviate from their typical or expected behavior.

To effectively detect anomalous user behavior, organizations can employ log analysis solutions that incorporate detection engines and machine learning algorithms to establish normal behavior patterns. Deviations from these patterns or baselines can then be alerted as potential security incidents. Some examples of these solutions include *Splunk User Behavior Analytics (UBA)*, *IBM QRadar UBA*, and *Azure AD Identity Protection*.

The specific indicators can vary greatly depending on the source, but some examples of this that can be found in log files include:

- **Multiple failed login attempts**
  - Unusually high numbers of failed logins within a short time may indicate a brute-force attack.
- **Unusual login times**
  - Login events outside the user's typical access hours or patterns might signal unauthorized access or compromised accounts.
- **Geographic anomalies**
  - Login events from IP addresses in countries the user does not usually access can indicate potential account compromise or suspicious activity.
  - In addition, simultaneous logins from different geographic locations (or indications of impossible travel) may suggest account sharing or unauthorized access.
- **Frequent password changes**
  - Log events indicating that a user's password has been changed frequently in a short period may suggest an attempt to hide unauthorized access or take over an account.

Task 5: Automatic vs Manual analysis



Task 5 Automated vs. Manual Analysis

Automated Analysis

Automated analysis involves the use of tools. For example, these often include commercial tools such as XPLG or SolarWinds Loggly. Automated analysis tools allow for processing and data analysis of logs. These tools often utilize Artificial Intelligence / Machine Learning to analyze patterns and trends. As the AI landscape evolves, we expect to see more effective automated analysis solutions.

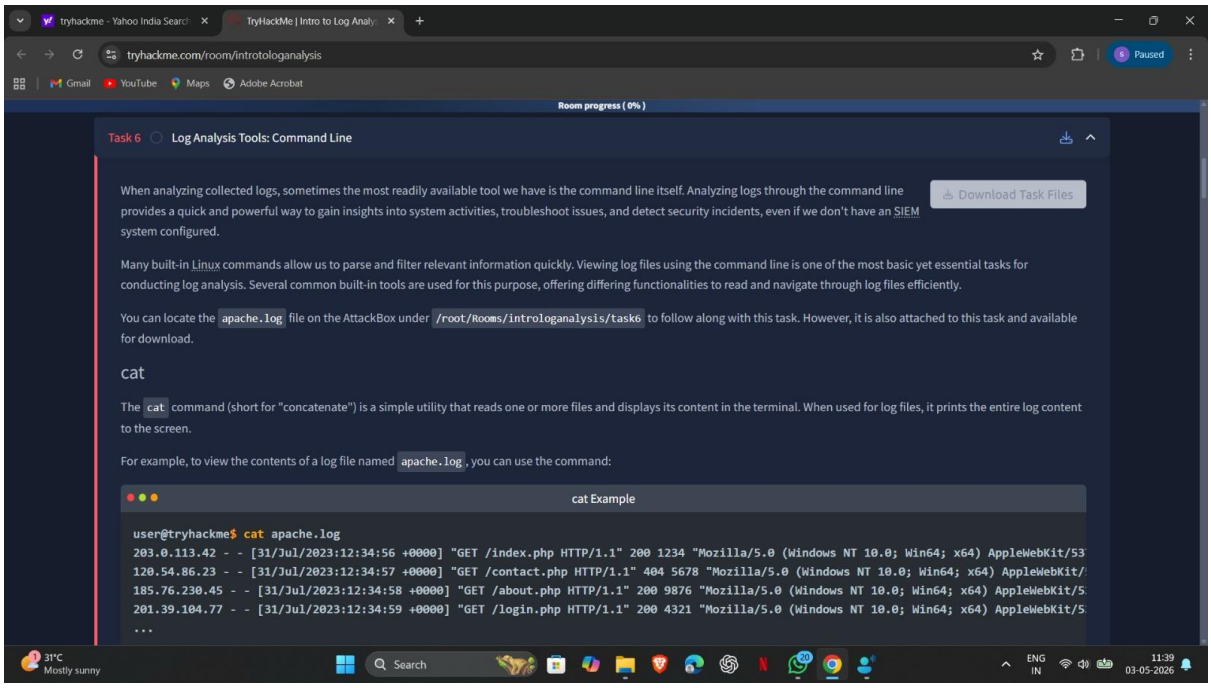
Advantages	Disadvantages
Saves time by performing a lot of the manual work required in manual analysis	Automated analysis tools are usually commercial-only and, therefore, expensive.
The use of artificial intelligence is effective at recognizing patterns and trends.	The effectiveness of artificial intelligence depends on how capable the model is. For example, the risk of false positives increases, or newer or never-seen-before events can be missed as the AI is not trained to recognize these.

Manual Analysis

Manual analysis is the process of examining data and artifacts without using automation tools. For example, an analyst scrolling through a web server log would be considered manual analysis. Manual analysis is essential for an analyst because automation tools cannot be relied upon.

Advantages	Disadvantages
It is cheap and does not require expensive tooling. For example, simple Linux commands can do the trick.	It is time-consuming as the analyst has to do all of the work, including reformatting log files.

Task 6: Log analysis: Tool Command Line



Task 6 Log Analysis Tools: Command Line

When analyzing collected logs, sometimes the most readily available tool we have is the command line itself. Analyzing logs through the command line provides a quick and powerful way to gain insights into system activities, troubleshoot issues, and detect security incidents, even if we don't have an SIEM system configured.

Download Task Files

Many built-in Linux commands allow us to parse and filter relevant information quickly. Viewing log files using the command line is one of the most basic yet essential tasks for conducting log analysis. Several common built-in tools are used for this purpose, offering differing functionalities to read and navigate through log files efficiently.

You can locate the `apache.log` file on the AttackBox under `/root/Rooms/introloganalysis/task6` to follow along with this task. However, it is also attached to this task and available for download.

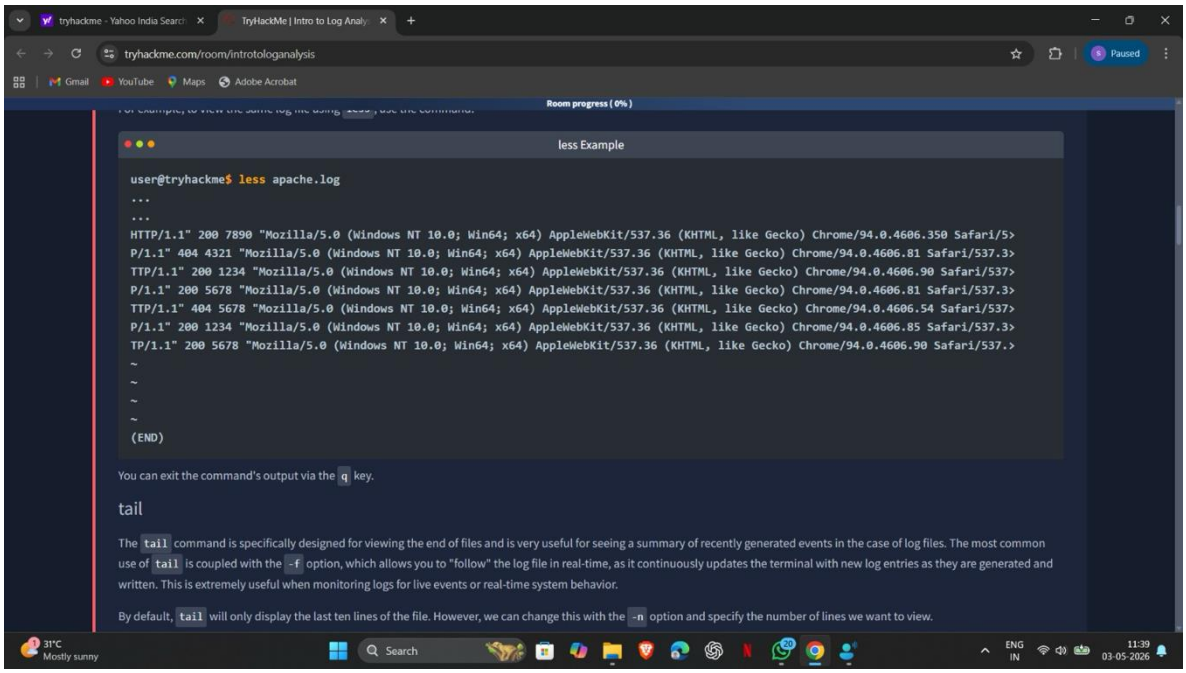
cat

The `cat` command (short for "concatenate") is a simple utility that reads one or more files and displays its content in the terminal. When used for log files, it prints the entire log content to the screen.

For example, to view the contents of a log file named `apache.log`, you can use the command:

cat Example

```
user@tryhackme$ cat apache.log
203.0.113.42 - - [31/Jul/2023:12:34:56 +0000] "GET /index.php HTTP/1.1" 200 1234 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/53
120.54.86.23 - - [31/Jul/2023:12:34:57 +0000] "GET /contact.php HTTP/1.1" 404 5678 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/
185.76.230.45 - - [31/Jul/2023:12:34:58 +0000] "GET /about.php HTTP/1.1" 200 9876 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/5
201.39.104.77 - - [31/Jul/2023:12:34:59 +0000] "GET /login.php HTTP/1.1" 200 4321 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/5
...
```



less Example

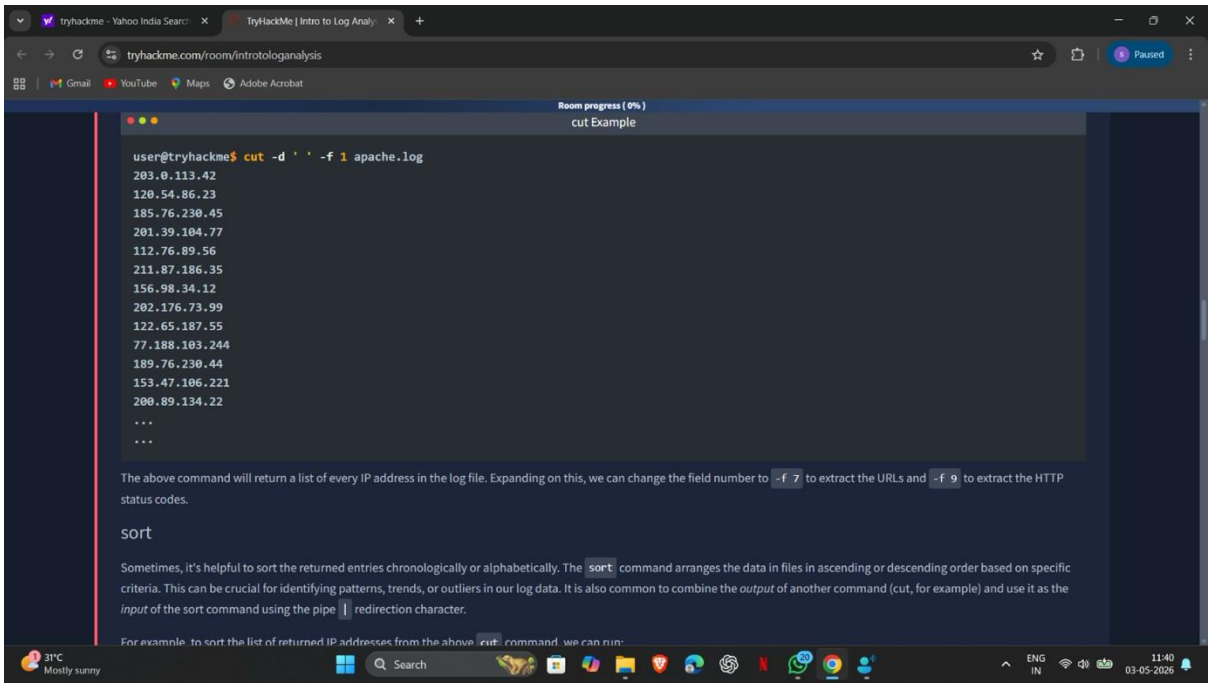
```
user@tryhackme$ less apache.log
...
...
HTTP/1.1" 200 7890 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/94.0.4606.350 Safari/5>
P/1.1" 404 4321 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/94.0.4606.81 Safari/537.3>
TTP/1.1" 200 1234 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/94.0.4606.90 Safari/537>
P/1.1" 200 5678 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/94.0.4606.81 Safari/537.3>
TTP/1.1" 404 5678 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/94.0.4606.54 Safari/537>
P/1.1" 200 1234 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/94.0.4606.85 Safari/537.3>
TP/1.1" 200 5678 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/94.0.4606.90 Safari/537.>
~
~
~
~
(END)
```

You can exit the command's output via the `q` key.

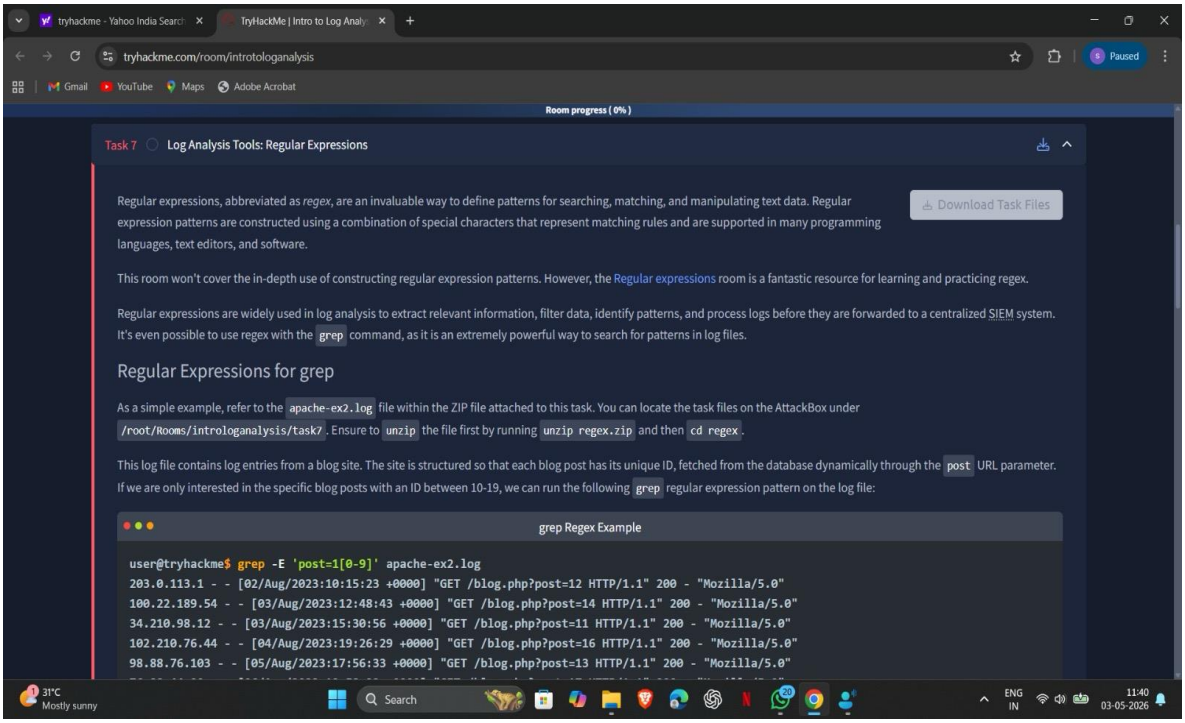
tail

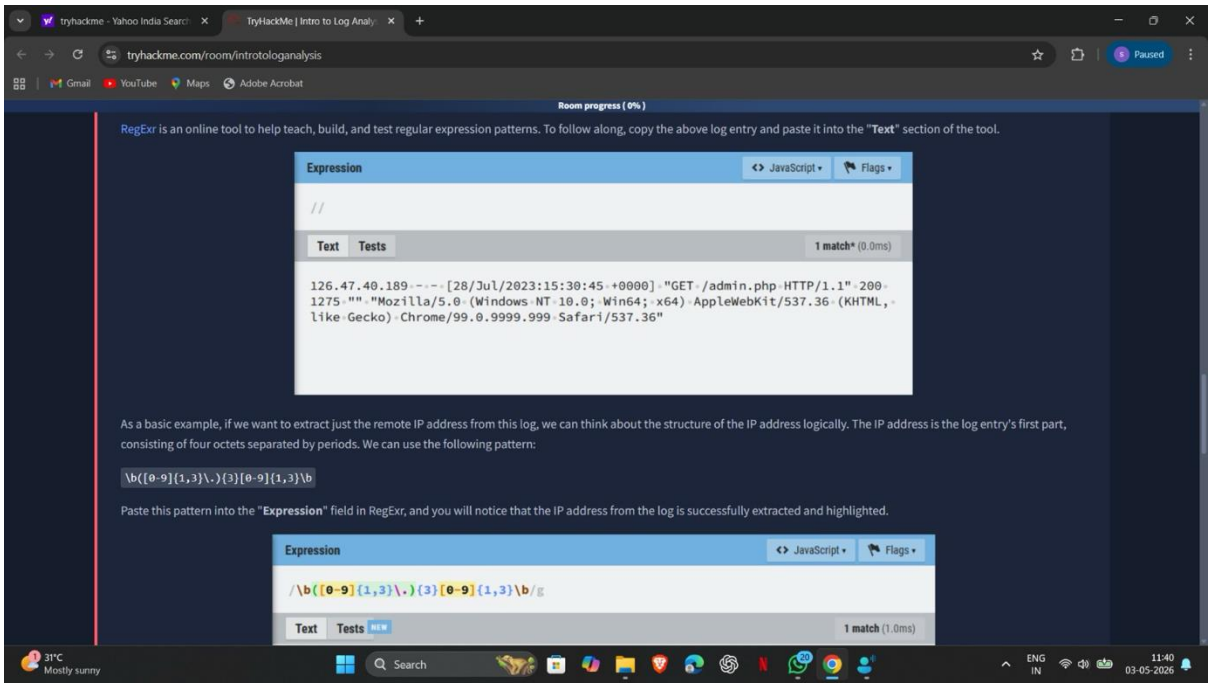
The `tail` command is specifically designed for viewing the end of files and is very useful for seeing a summary of recently generated events in the case of log files. The most common use of `tail` is coupled with the `-f` option, which allows you to "follow" the log file in real-time, as it continuously updates the terminal with new log entries as they are generated and written. This is extremely useful when monitoring logs for live events or real-time system behavior.

By default, `tail` will only display the last ten lines of the file. However, we can change this with the `-n` option and specify the number of lines we want to view.

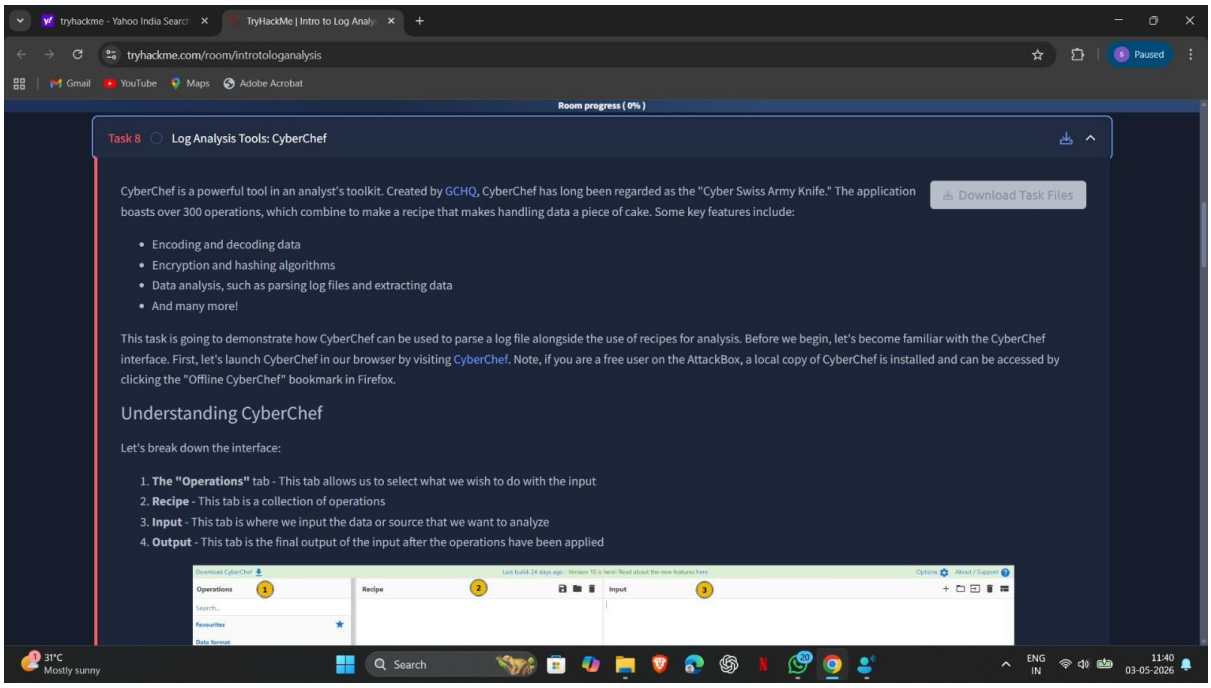


Task 7: Log Analysis Tool: Regular Expressions





Task 8: Log Analysis Tool: CyberChef



tryhackme - Yahoo India Search | TryHackMe | Intro to Log Analy

tryhackme.com/room/introtologanalysis

Gmail YouTube Maps Adobe Acrobat

Room progress (0%)

Task 8 Log Analysis Tools: CyberChef

Download Task Files

CyberChef is a powerful tool in an analyst's toolkit. Created by GCHQ, CyberChef has long been regarded as the "Cyber Swiss Army Knife." The application boasts over 300 operations, which combine to make a recipe that makes handling data a piece of cake. Some key features include:

- Encoding and decoding data
- Encryption and hashing algorithms
- Data analysis, such as parsing log files and extracting data
- And many more!

This task is going to demonstrate how CyberChef can be used to parse a log file alongside the use of recipes for analysis. Before we begin, let's become familiar with the CyberChef interface. First, let's launch CyberChef in our browser by visiting CyberChef. Note, if you are a free user on the AttackBox, a local copy of CyberChef is installed and can be accessed by clicking the "Offline CyberChef" bookmark in Firefox.

Understanding CyberChef

Let's break down the interface:

1. **The "Operations"** tab - This tab allows us to select what we wish to do with the input
2. **Recipe** - This tab is a collection of operations
3. **Input** - This tab is where we input the data or source that we want to analyze
4. **Output** - This tab is the final output of the input after the operations have been applied

31°C Mostly sunny Search ENG IN 11:40 03-05-2026

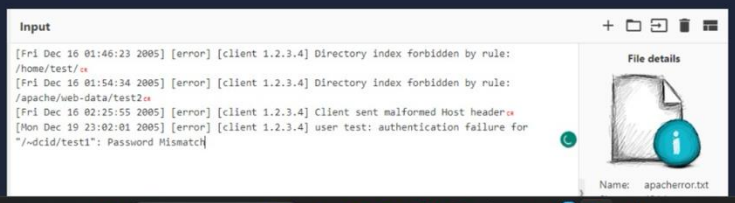
tryhackme - Yahoo India Search | TryHackMe | Intro to Log Analy

tryhackme.com/room/introtologanalysis

Gmail YouTube Maps Adobe Acrobat

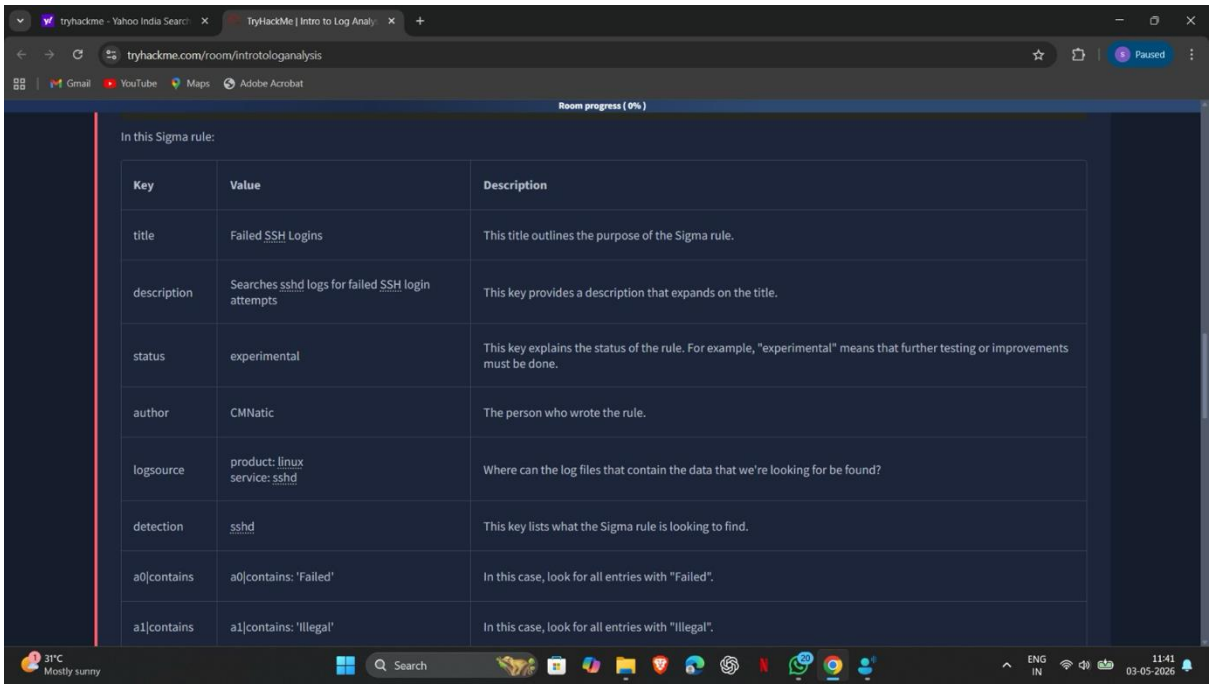
Room progress (0%)

Files and folders can be uploaded to CyberChef. This provides a convenient way of uploading log files to CyberChef. To do so, click on the box with an arrow pointing inside it. Additionally, CyberChef has operators that allow you to unzip compressed files, such as *.tar.gz* or *.zip*.



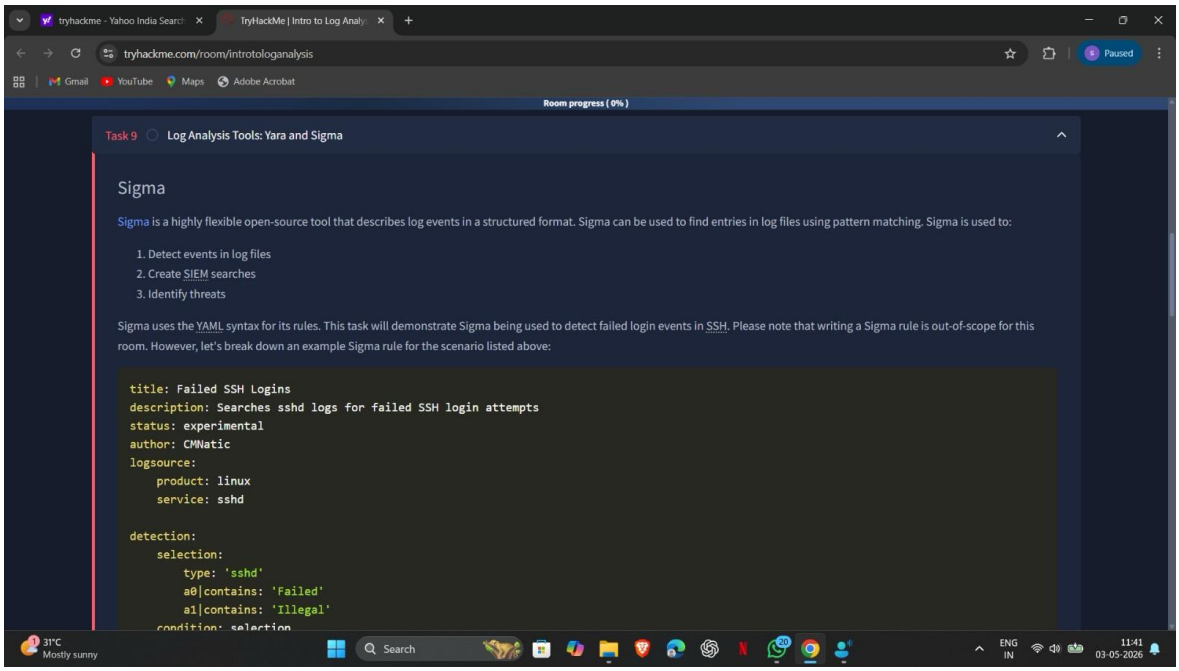
31°C Mostly sunny Search ENG IN 11:41 03-05-2026

Task 9: Log Analysis Tool: Yara and Sigma



In this Sigma rule:

Key	Value	Description
title	Failed SSH Logins	This title outlines the purpose of the Sigma rule.
description	Searches sshd logs for failed SSH login attempts	This key provides a description that expands on the title.
status	experimental	This key explains the status of the rule. For example, "experimental" means that further testing or improvements must be done.
author	CMNatic	The person who wrote the rule.
logsource	product: linux service: sshd	Where can the log files that contain the data that we're looking for be found?
detection	sshd	This key lists what the Sigma rule is looking to find.
a0|contains	a0|contains: 'Failed'	In this case, look for all entries with "Failed".
a1|contains	a1|contains: 'Illegal'	In this case, look for all entries with "Illegal".



Task 9 Log Analysis Tools: Yara and Sigma

Sigma

Sigma is a highly flexible open-source tool that describes log events in a structured format. Sigma can be used to find entries in log files using pattern matching. Sigma is used to:

1. Detect events in log files
2. Create SIEM searches
3. Identify threats

Sigma uses the YAML syntax for its rules. This task will demonstrate Sigma being used to detect failed login events in SSH. Please note that writing a Sigma rule is out-of-scope for this room. However, let's break down an example Sigma rule for the scenario listed above:

```
title: Failed SSH Logins
description: Searches sshd logs for failed SSH login attempts
status: experimental
author: CMNatic
logsource:
    product: linux
    service: sshd

detection:
    selection:
        type: 'sshd'
        a0|contains: 'Failed'
        a1|contains: 'Illegal'
    condition: selection
```

Result:

Successfully analyzed system and network logs to identify suspicious activities and potential security incidents. Gained practical knowledge in detecting Indicators of Compromise (IoCs) and responding to threats using log analysis techniques in a SOC environment.